

event to happen. After the given amount of time, pygame will issue a timer event in the main loop, in the same way that it issues other standard events such as `KEYUP`, `KEYDOWN`, `MOUSEBUTTONUP`, `MOUSEBUTTONDOWN`, and so on. Your code will need to look for and react to this type of event.

The following documentation is from <https://www.pygame.org/docs/ref/time.html>:

```
pygame.time.set_timer()
```

Repeatedly create an event on the event queue

```
set_timer(eventid, milliseconds) -> None
```

```
set_timer(eventid, milliseconds, once) -> None
```

Set an event type to appear on the event queue every given number of milliseconds. The first event will not appear until the amount of time has passed.

Every event type can have a separate timer attached to it. It is best to use the value between `pygame.USEREVENT` and `pygame.NUMEVENTS`.

To disable the timer for an event, set the `milliseconds` argument to 0.

If the `once` argument is `True`, then only send the timer once.

Every event type in pygame is represented by unique identifier. As of pygame 2.0, you can now make a call to `pygame.event.custom_type()` to get an identifier for a custom event.

File: `InLineTimerExamples /TimerEvent.py`

```
TIMER_EVENT_ID = pygame.event.custom_type() # new in pygame 2.0
TIMER_LENGTH = 2.5 # seconds
```

When the user clicks Start, the code creates and posts the timer event:

```
if startButton.handleEvent(event):
    pygame.time.set_timer(TIMER_EVENT_ID,
                          int(TIMER_LENGTH * 1000), True)
    --- snip disable button, show message ---
```
